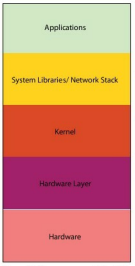

Figure 1: Components of the IoT

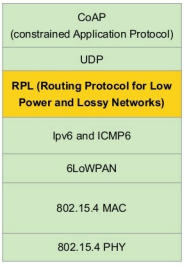

Figure 2: Network stack for IoT

Comparisons with Contiki OS and Tiny OS

There are two other OSs, called Contiki OS (already featured in *OSFY*) and Tiny OS, which are suitable for IoT applications. But RIOT fares better when it comes to memory usage and support.

Table 1 gives a comparision between these OSs.

Features	Contiki OS	Tiny OS	RIOT OS
Minimum RAM and ROM needed	<2KB, <30KB	<1KB, <4KB	~1.5KB, '5KB
C and C++ support	Partial	No support (nesC Compiler)	Full support
Multi-threading	No (proto threads are used here)	No	Yes
Real-time capabilities	No	No	Yes

Table 1: A comparison of various OSs for their IoT suitability

Figures 1 and 2 represent the block and network stack diagram of the IoT. Except for RPL, all modules have support from the RIOT OS (for RPL alone, there is partial support from RIOT).

Installation of RIOT in Linux

The OS used for installation is Linux Mint 17.2 and Ubuntu 14.04.2, while the architecture used is 64-bit OSs. The pre-requisite packages for installing RIOT are:

- msp 430 GCC compiler
- msp 430 libraries
- avr utils
- arm based gcc
- gcc multi-library, etc

Assuming that you have installed all just now, follow the steps shown below to install RIOT:

```
pradeepkumar $] sudo apt-get update
pradeepkumar $] sudo apt-get install libc6-dev-i386 build-essential binutils-msp430 gcc-msp430 msp430-libc libncurses5-dev openjdk-7-jre openjdk-7-jdk avrdude avr-libc gcc-avr gdb-avr binutils-avr mspdebug msp430mcu autoconf automake libxmu-dev gcc-arm-none-eabi openocd git bridge-utils gcc-multilib socket
```

The OS can be downloaded as a zip file from Github (*https://github.com/RIOT-OS/RIOT/archive/master.zip*); else, you can clone it using the following command:

```
pradeepkumar $] git clone https://github.com/RIOT-OS/RIOT.git
```

The folder *RIOT/* or *RIOT-Master/* contains various sub-folders:

->boards: This folder has all the drivers for the boards that are supported by the OS

->cpu: This folder contains the architecture supported by RIOT

->cores: This has the core of the OS, like kernel, modules, etc

->doc: This folder contains documentation

->examples: Some examples like 'hello world', border router, Ipv6 formation, etc

->drivers: This one has drivers' for all the sensors, etc

A simple example

These types of OSs can be learnt and understood only through the examples given within the source code of the application. RIOT has some examples in the *examples/* folder, and one can go through the source code and run it using the following method. There are more examples available in the Github of RIOT (*https://github.com/RIOT-OS).*

If sufficient board or hardware is not available, then the code can be run in the terminal itself (native mode).

To run the *'hello-world'* example from the *examples* folder, use the following code:

```
pradeepkumar $] cd RIOT-Master/examples/hello-world
pradeepkumar $] make flash
pradeepkumar $] make term
-----OUTPUT-----
/home/pradeepkumar/RIOT-master/examples/hello-world/bin/native/hello-world.elf
RIOT native interrupts/signals initialized.
LED_GREEN_OFF
LED_RED_ON
RIOT native board initialized.
RIOT native hardware initialization complete.

main(): This is RIOT! (Version: UNKNOWN (builddir: /home/pradeepkumar/RIOT-master))
Hello World!
```